

# 7장 프로젝트 리팩터링과 배포

프로젝트 리팩터링과 배포 장은 기존 코드베이스를 안전하고 점진적으로 개선하는 리팩토링 기법(중복 제거, 공통 컴포넌트·유틸 추출, 상태 재배치, useReducer/Context 도입 등)과 코드 품질 파이프라인(ESLint/Prettier, husky, CI) 구축을 통해 협업·유지보수성을 높이는 방법을 배우고, 이어서 빌드·배포 설정(번들 최적화·환경변수·Netlify/Vercel·CD 파이프라인)과 운영 모니터링까지 연결해 실제 서비스 운영에 필요한 전 과정을 실무 관점으로 익히게 한다.

## 7.1 코드 리팩토링 기법

이번 절에서는 반복되는 코드 구조를 제거하고 공통 컴포넌트화하는 방법, 일관된 네이밍 규칙과 폴더 구조 정리, 상태 관리 위치 이동 및 분리 전략을 통해 코드 품질을 향상시키는 기법을 학습하며, 서비스 규모 확장 시에도 유지보수가 용이한 리액트 애플리케이션을 구축하는 방법을 배운다.

### (1) 반복 구조 제거 및 공통 컴포넌트화

비슷한 UI나 기능을 반복 작성하지 않고 공통 컴포넌트로 분리하여 코드 재사용성과 유지보수를 높이는 리팩토링 기법을 다룬다.

#### ① 반복 식별

반복 구조는 UI(버튼·카드·로더 등)와 로직(fetch, 포매팅, 에러 처리 등) 양쪽에서 나오므로 변경 빈도·복잡도·테스트 필요성으로 우선순위를 정해 추출 대상을 식별하고, 소규모의 안전한 리팩토링 단위로 분리하는 것이 바람직하다.

- **목표:** 중복 코드를 공통 컴포넌트(또는 훅)로 추출해 재사용성·테스트성 향상.
- **프로세스:** (1) 반복 코드를 찾는다 → (2) props/인터페이스 설계 → (3) 테스트 추가 → (4) 교체(리팩토링) 및 회귀 검증.

#### 중복 버튼 추출 전/후

중복된 버튼들이 여러 파일에 있을 때(추출 전):

```
// A.jsx
<button className="btn" onClick={onSave}>저장</button>
```

```
// B.jsx
<button className="btn primary" onClick={onDelete}>삭제</button>
```

추출 후 `src/components/ui/Button.jsx` :

```
// Button.jsx
import React from 'react';
export default function Button({ children, variant='primary', className='',
...props }){
  const cls = `btn ${variant} ${className}`.trim();
  return <button className={cls} {...props}>{children}</button>;
}
```

사용 시

```
import Button from '../components/ui/Button';
<Button onClick={onSave}>저장</Button>
<Button variant="danger" onClick={onDelete}>삭제</Button>
```

- 추출 후 스타일/공통 동작(비활성화, 접근성, 로딩 표시 등)을 하나의 컴포넌트에서 관리.
- 단위 테스트: Button에 대한 렌더/클릭 테스트만으로 여러 위치의 동작 검증 가능.

## ② 로직 반복 혹은 유틸화

네트워크 호출·포맷 변환·로컬스토리지 동기화 같은 반복 로직은 커스텀 훅( `useFetch` , `useLocalStorage` )이나 유틸 함수( `formatPrice` , `apiClient` )로 분리해 컴포넌트에서는 선언적 사용만 하도록 하여 비즈니스 로직과 렌더링을 분리한다.

`useLocalStorage` 훅 예제

```
// src/hooks/useLocalStorage.js
import { useState, useEffect } from 'react';
export default function useLocalStorage(key, initial){
  const [state, setState] = useState(() => {
    const s = localStorage.getItem(key);
    return s ? JSON.parse(s) : initial;
  });
```

```
useEffect(() => { localStorage.setItem(key, JSON.stringify(state)); }, [key, state]);
return [state, setState];
}
```

사용:

```
const [todos, setTodos] = useLocalStorage('todos', []);
```

### ③ 추출 인터페이스 설계와 테스트

추출 시 가장 중요한 것은 명확한 인터페이스(Props, 반환값) 설계로, 이는 문서화와 타입 (PropTypes/TypeScript)으로 보완하고 단위 테스트를 추가해 리팩토링 이후에도 동작 보장을 확보한다.

#### 권장 절차

- Props 최소화: `onAction`, `label`, `disabled` 등 공통 contract만 노출.
- TypeScript/PropTypes로 타입 계약을 강제.
- Jest + React Testing Library로 핵심 행위(클릭, disabled, 렌더링 텍스트) 테스트 작성

## (2) 네이밍 룰 및 폴더 정리

변수, 함수, 컴포넌트의 이름 규칙을 일관성 있게 적용하고, 폴더를 기능 또는 도메인 단위로 정리해 협업과 가독성을 개선하는 기법을 학습한다.

### ① 컴포넌트·파일 네이밍 규칙

컴포넌트는 `PascalCase` 파일명과 기본 `export`를 일치시키고 혹은 `use` 접두사로, 유틸은 소문자 카멜(`formatPrice`)로 통일하는 규칙을 팀 표준으로 문서화하여 신규 기여자의 혼란을 줄인다.

#### 예

- `src/components/Header.jsx` → `export default Header`
- `src/hooks/useAuth.js` → `export default function useAuth(){...}`

## ② 폴더 구조 권장

대형 프로젝트에서는 `domains/` (혹은 `features/`)를 루트로 두고 공통 UI는 `components/ui/`, 페이지는 `pages/`, 서비스(API)는 `services/`로 분리하는 도메인 중심 구조가 유지보수에 유리하다.

### 샘플 구조

```
src/  
├─ domains/  
│   └─ product/  
│       ├─ ProductList.jsx  
│       ├─ api.js  
│       └─ styles.module.css  
├─ components/  
│   └─ ui/  
│       ├─ Button.jsx  
│       └─ Input.jsx  
├─ pages/  
├─ services/  
└─ contexts/
```

## ③ export 패턴과 진입점

각 도메인 폴더에 `index.js` (re-export)를 두어 외부에서 `import { ProductList } from 'domains/product'`처럼 단일 진입점으로 접근하게 하면 리팩토링 시 내부 파일 이동으로 인한 импорт 변경 범위를 줄일 수 있다.

### 코드 — re-export 예

```
// src/domains/product/index.js  
export { default as ProductList } from './ProductList';  
export { default as ProductDetail } from './ProductDetail';
```

사용:

```
import { ProductList } from 'domains/product';
```

### (3) 상태 위치 이동 및 분리 전략

상태 관리가 불필요하게 중첩된 경우 상위 컴포넌트로 이동하거나 Context·전역 상태 관리 도구로 분리해 컴포넌트 간 의존성을 줄이는 방법을 다룬다.

#### ① 로컬 vs 전역 상태 판정

상태를 전역으로 올릴지 여부는 “여러 컴포넌트가 공유하는가?”와 “업데이트 빈도가 높은가?”를 기준으로 판단하며, 공유 필요가 없다면 로컬(컴포넌트)로 남겨 복잡도를 낮추는 것이 원칙이다.

- Form 입력, UI 토글 등은 로컬.
- 인증, 카트, 테마 등은 전역.

#### ② 상태 끌어올리기(Lift state up)

하위 컴포넌트들 간에 동일한 데이터가 필요하면 공통 조상으로 상태를 올리고 변경 함수를 props로 전달하는 전통적 패턴을 사용하되, 전달 단계가 너무 길어지면 Context나 전역 상태 도구로 대체한다.

```
// Parent.jsx
const [value, setValue] = useState("");
<ChildA value={value} onChange={setValue}/>
<ChildB value={value}/>
```

#### ③ Context vs 전역 상태 라이브러리

Context는 소규모 전역 상태(테마, auth)에 적합하며, 복잡한 비동기/캐시/성능 요구(여러 곳에서 빈번 업데이트, 비동기 캐시 등)에는 react-query나 Redux/Zustand 등을 고려해 사용하라.

##### 권장 기준

- 단순 공유: Context
- 캐시/비동기/서버 상태: react-query
- 복잡한 전역 상태(복합 액션): Redux/Zustand

#### ④ useReducer 패턴

하나의 컴포넌트 또는 도메인에서 상태 로직이 복잡해질 때 `useReducer` 로 액션 패턴을 도입하면 상태 변경이 예측 가능하고 테스트하기 쉬운 구조가 된다.

##### useReducer 장바구니

```
const initial = [];  
function reducer(state, action){  
  switch(action.type){  
    case 'ADD': return [...state, action.payload];  
    case 'REMOVE': return state.filter(i⇒i.id!==action.payload);  
    default: return state;  
  }  
}  
  
const [items, dispatch] = useReducer(reducer, initial);  
dispatch({ type:'ADD', payload: product });
```

## (4) 코드 품질 향상 실습

ESLint, Prettier 등을 활용해 일관된 코드 스타일을 유지하고, 불필요한 의존성 제거와 최적화를 통해 전반적인 코드 품질을 향상시키는 실습을 진행한다.

#### ① ESLint + Prettier 설정

ESLint(코드 품질 규칙)와 Prettier(코드 포맷)를 함께 사용하고 프로젝트 루트에 설정 파일을 두며, `npm run lint` · `npm run format` 스크립트를 package.json에 등록해 자동화한다.

`.eslintrc.cjs`

```
module.exports = {  
  env: { browser: true, es2021: true },  
  extends: ['eslint:recommended', 'plugin:react/recommended', 'plugin:react-hooks/recommended', 'prettier'],  
  parserOptions: { ecmaFeatures: { jsx: true }, ecmaVersion: 2021, sourceType: 'module' },  
  settings: { react: { version: 'detect' } },
```

```
rules: {}  
};
```

.prettierrc

```
{ "printWidth": 100, "singleQuote": true, "trailingComma": "es5" }
```

package.json scripts:

```
"scripts": {  
  "lint": "eslint 'src/**/*.js,jsx,ts,tsx'",  
  "format": "prettier --write 'src/**/*.js,jsx,ts,tsx,json,css,md'",  
}
```

## ② pre-commit 훅(husky + lint-staged)

Husky와 lint-staged를 사용해 커밋 전에 변경 파일만 대상으로 린트·포맷 검사를 실행하면 코드 스타일을 강제하면서 개발 흐름을 방해하지 않도록 할 수 있다.

### 설치·설정

```
npm install -D husky lint-staged  
npx husky install  
npx husky add .husky/pre-commit "npx lint-staged"
```

package.json

```
"lint-staged": {  
  "src/**/*.js,jsx,ts,tsx": ["eslint --fix", "prettier --write"]  
}
```

## ③ CI 파이프라인 예(GitHub Actions)

Pull Request마다 자동으로 `npm ci`, `npm run lint`, `npm test`, `npm run build`를 실행하는 간단한 GitHub Actions 워크플로우로 코드 품질과 빌드 안정성을 보장한다.

간단 워크플로우 ( [.github/workflows/ci.yml](#) )

```

name: CI
on: [pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '18' }
      - run: npm ci
      - run: npm run lint
      - run: npm test --if-present
      - run: npm run build

```

#### ④ 테스트·커버리지

React Testing Library와 Jest를 사용해 핵심 컴포넌트(폼, 버튼, 라우팅 흐름 등)의 동작 테스트를 작성하고, CI에서 커버리지 기준을 검증하여 기능 회귀를 예방한다.

##### 테스트 스니펫 (LoginForm 예)

```

// LoginForm.test.jsx (RTL)
import { render, screen, fireEvent } from '@testing-library/react';
import LoginForm from './LoginForm';
test('renders and submits', () => {
  const handle = jest.fn();
  render(<LoginForm onSuccess={handle} />);
  fireEvent.change(screen.getByPlaceholderText(/아이디/), { target: { value: 'alice' } });
  fireEvent.change(screen.getByPlaceholderText(/비밀번호/), { target: { value: 'pw' } });
  fireEvent.click(screen.getByRole('button', { name: /로그인/ }));
  expect(handle).not.toHaveBeenCalled(); // 실제 네트워크는 mocking 필요
});

```

(실제 네트워크는 `msw` 로 mock 처리 권장)



## 7.2 빌드 및 배포 프로세스

이번 절에서는 CRA와 Vite 기반의 빌드 방식을 이해하고, Netlify와 Vercel과 같은 클라우드 플랫폼을 활용한 배포 방법을 학습하며, CI 환경변수 처리와 배포 후 모니터링 흐름까지 경험하여 실제 서비스 운영에 필요한 전 과정의 배포 실무 능력을 익힌다.

### (1) CRA, Vite 기반 빌드 방식 이해

Create React App과 Vite를 사용해 애플리케이션을 빌드하는 과정을 비교하며 성능 최적화와 빌드 속도 차이를 이해한다.

#### ① 개발 서버와 빌드 커맨드 차이

CRA(webpack 기반)와 Vite(ESBuild/Rollup 기반)의 개발 서버 동작과 빌드 커맨드는 근본적으로 다르며, Vite는 개발 중 빠른 HMR과 빌드 전 처리 속도 이점이 있고 CRA는 안정된 플러그인·생태계 장점이 있어 프로젝트 요구에 따라 선택한다.

#### 명령

- CRA: `npx create-react-app myapp` → `npm run start` (dev), `npm run build` (prod)
- Vite: `npm create vite@latest myapp --template react` → `npm run dev`, `npm run build`
- CRA: webpack dev server, 번들링이 개발/빌드 모두 webpack 기반.
- Vite: 개발은 네이티브 ESM + esbuild로 빠르게, production 빌드는 Rollup으로 수행.

#### ② 번들러·트랜스파일러 차이와 최적화 포인트

CRA는 webpack과 Babel 중심으로 트랜스파일링·번들링을 처리해 다양한 플러그인과 로더를 활용하지만 빌드 속도가 느릴 수 있고, Vite는 esbuild로 개발 중 빠른 트랜스파일을 제공하고 프로덕션 빌드는 Rollup으로 번들링하여 최종 번들 크기와 성능 최적화에 유리하다.

#### 권장 최적화 체크리스트

- 불필요한 폴리필 제거(타깃 브라우저 명시)

- 라이브러리의 ESM 번들 사용(예: `module` 필드)
- tree-shaking 가능한 코드 스타일(임포트 사용 방식)
- 의존성 번들링 최적화(large deps 외부화)

### ③ 코드 스플리팅과 레이저 로딩

라우트 단위나 컴포넌트 단위로 `React.lazy` 와 `Suspense` 또는 동적 `import()` 를 사용해 코드 스플리팅을 적용하면 초기 번들 크기를 줄여 첫 로드 성능을 개선할 수 있다.

#### React lazy + Suspense

```
// routes.js
import React, { Suspense, lazy } from 'react';
const Home = lazy(() => import('./pages/Home'));
const Product = lazy(() => import('./pages/Product'));

export default function AppRoutes(){
  return (
    <Suspense fallback=<div>로딩중...</div>>
      { /* Routes... */ }
    </Suspense>
  );
}
```

Vite/CRA 모두에서 동적 `import()`로 청크가 분리됨. 서버에선 HTTP/2 혹은 CDN 캐싱 정책을 신경써야 효과적.

### ④ 번들 분석 및 용량 감소 방안

번들 시각화 도구(예: `rollup-plugin-visualizer` 혹은 `source-map-explorer`)로 의존성별 용량을 파악하고, 대형 라이브러리 교체(예: `lodash` 개별 임포트, `date-fns` 사용), 이미지 압축, 코드스플리팅으로 번들 크기를 체계적으로 줄인다.

#### Vite + visualizer

```
// vite.config.js (간단)
import { defineConfig } from 'vite';
import visualizer from 'rollup-plugin-visualizer';
export default defineConfig({
```

```
plugins: [visualizer({ open: true })]
});
```

시각화 결과를 보면서 큰 모듈을 트리 세이킹할 수 있는지, CDN으로 외부화할지 결정.

## (2) 배포 환경 설정 (Netlify, Vercel)

무료로 제공되는 Netlify와 Vercel 플랫폼을 활용해 리액트 프로젝트를 손쉽게 배포하고, 도메인 연결과 HTTPS 지원을 설정하는 방법을 다룬다.

### ① Netlify 빠른 배포 및 SPA 폴백 설정

Netlify는 Git 연결만으로 자동빌드·배포가 가능하며 SPA의 브라우저 라우팅을 위해 `_redirects` 또는 `netlify.toml` 에 모든 경로를 `index.html` 로 포워딩하도록 설정하면 클라이언트 라우팅 문제를 방지할 수 있다.

파일 예: `public/_redirects`

```
/* /index.html 200
```

### Netlify 기본 배포 절차

1. GitHub 리포지토리 연결
2. Build command: `npm run build` , Publish dir: `dist` (Vite) 또는 `build` (CRA)
3. 환경변수(Env) 설정(사이트 설정 → Build & deploy → Environment)

### ② Netlify 고급 설정

`netlify.toml` 에 빌드 설정, 리다이렉트, 헤더 설정을 선언적(파일)으로 관리하면 팀 단위로 배포 규칙을 코드베이스에 포함시킬 수 있고, 브랜치별 배포 설정도 손쉽게 관리할 수 있다.

예: `netlify.toml`

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
```

```
from = "/*"  
to = "/index.html"  
status = 200
```

브랜치별 deploy 설정 추가 가능, 커스텀 헤더 설정으로 보안 정책 적용 가능.

### ③ Vercel 배포 및 설정

Vercel은 프론트엔드에 특화된 배포 플랫폼으로 Git 연동 후 자동 배포, 프리뷰 URL을 제공하며 `vercel.json` 로 리writes-headers 등을 설정하거나 대시보드에서 환경변수를 관리해 빠른 CI/CD를 할 수 있다.

`vercel.json`

```
{  
  "builds": [{ "src": "package.json", "use": "@vercel/static-build", "config": {  
    "distDir": "dist" } }],  
  "routes": [{ "src": "/(.*)", "dest": "/index.html" }]  
}
```

Vercel은 기본으로 HTTPS, CDN 배포를 제공하고 Serverless Functions를 통합해 런타임 API를 같이 배포 가능.

### ④ 커스텀 도메인과 HTTPS 설정

Netlify/Vercel 모두 커스텀 도메인을 연결하면 자동으로 Let's Encrypt 기반의 HTTPS를 발급해 주며, DNS 레코드(A/ALIAS/CNAME) 설정을 통해 도메인 연결을 완료하고 1~24 시간 내에 인증서를 발급받을 수 있다.

#### 설정 포인트

- Netlify: 사이트 → Domain management → Add custom domain → DNS provider에 CNAME/A 레코드 추가
  - Vercel: Project → Domains → Add Domain → DNS 검사 → 자동 인증서 발급
- 주의: `www` 와 루트 도메인 모두를 설정하고 리디렉션 정책(예: 루트→www)을 정함.

## (3) CI 환경변수 처리 및 연결

API 키나 비밀 값을 환경변수로 설정하고 CI 파이프라인에 반영해 보안성을 확보하며, 자동 배포와 연계하는 실무 흐름을 학습한다.

## ① 빌드타임 vs 런타임 환경변수

환경변수는 빌드 타임(번들에 하드코딩)과 런타임(서버/edge에서 읽힘)으로 나뉘므로 민감한 시크릿은 빌드 번들에 포함시키지 말고 서버사이드나 런타임 함수로 프록시해야 보안상 안전하다; CRA는 `REACT_APP_` 접두사, Vite는 `VITE_` 접두사를 사용한다.

예

- CRA: `.env.production` → `REACT_APP_API_URL=https://api.example.com`
- Vite: `.env.production` → `VITE_API_URL=https://api.example.com`

주의: 이 값들은 빌드 시점에 번들에 포함됨 — 절대 비밀값(토큰)은 넣지 말 것.

## ② GitHub Actions에 시크릿 연결

CI에서 민감한 값은 GitHub Secrets에 저장하고 워크플로우에서 `${{ secrets.MY_SECRET }}` 로 주입해 빌드 단계에서만 사용하며, public 로그에 노출되지 않도록 주의한다.

### GitHub Actions snippet

```
name: Deploy
on: push:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build
      env:
        VITE_API_URL: ${ secrets.VITE_API_URL }
      - uses: peaceiris/actions-gh-pages@v3
      with:
        publish_dir: ./dist
```

secrets는 GitHub 리포지토리 Settings → Secrets에 저장.

### ③ 런타임 시크릿과 서버리스 활용

클라이언트에 절대 노출하면 안 되는 시크릿(예: 서드파티 비밀키)은 서버리스 함수(Netlify Functions / Vercel Serverless / AWS Lambda) 내에서만 사용하고, 클라이언트는 해당 함수에 요청하여 간접적으로 안전하게 기능을 수행하게 한다.

#### Netlify Function

```
// netlify/functions/proxy.js
exports.handler = async function(event) {
  const secret = process.env.SECRET_API_KEY; // CI에 입력된 시크릿
  // 외부 API 호출...
  return { statusCode: 200, body: JSON.stringify({ok:true}) };
}
```

이 함수는 빌드/런타임 환경변수로 시크릿을 안전하게 갖고 있음.

### ④ .env 파일 관리 및 보안 규칙

프로젝트 루트의 `.env` 파일은 절대 깃에 커밋하지 말고 `.gitignore`에 추가하며, `.env.example`으로 필요한 키 목록과 형식을 문서화해 팀원이 로컬 환경을 구성할 수 있도록 한다.

#### 권장 파일

```
.env.example
REACT_APP_API_URL=https://api.example.com
VITE_API_URL=https://api.example.com
```

주의: 실제 값은 CI/플랫폼 시크릿으로 관리.

## (4) 배포 후 모니터링 흐름 이해

실제 운영 환경에서 성능 모니터링과 로그 추적을 통해 오류를 빠르게 파악하고 안정적인 서비스 운영을 유지하는 모니터링 전략을 학습한다.

### ① 클라이언트 오류 모니터링

실사용에서 발생하는 런타임 에러를 자동 수집하려면 Sentry 같은 오류 모니터링 도구를 도입해 크래시/에러 스택, 유저 컨텍스트, 브라우저 환경을 전송하고 알림 규칙을 설정하면 문제 발생 시 빠르게 대응할 수 있다.

## Sentry 초기화 (React)

```
// src/sentry.js
import * as Sentry from "@sentry/react";
Sentry.init({ dsn: process.env.VITE_SENTRY_DSN, integrations: [] });
```

DSN은 플랫폼에 설정된 시크릿으로 관리. `Sentry.captureException(err)` 로 수동 전송 가능.

## ② 성능 모니터링 (Web Vitals)

사용자 경험 개선을 위해 `web-vitals` 라이브러리로 LCP, FID, CLS 같은 핵심 지표를 수집하여 서버로 전송하거나 APM(예: New Relic, Datadog)으로 집계하면 페이지 성능 병목을 식별·개선할 수 있다.

## CRA 기본 템플릿 방식

```
// src/reportWebVitals.js (간단)
import { getCLS, getFID, getLCP } from 'web-vitals';
export function reportWebVitals(onPerfEntry){
  if(onPerfEntry && onPerfEntry instanceof Function){
    getCLS(onPerfEntry); getFID(onPerfEntry); getLCP(onPerfEntry);
  }
}
// index.js
import { reportWebVitals } from './reportWebVitals';
reportWebVitals(console.log);
```

console 대신 전송 API로 서버에 전송해 모니터링 대시보드 구성.

## ③ 로깅·애널리틱스 통합

클라이언트 이벤트(사용자 행동)와 서버 로그를 연계해 문제 재현성을 높이고, 로그는 JSON 형식으로 중앙 로그 수집(ELK, Datadog)으로 모아 검색·알림을 설정하면 운영 효율이 증가한다.

## 이벤트 전송

```
// 트래킹 helper
export function trackEvent(name, payload){
  navigator.sendBeacon('/.netlify/functions/track', JSON.stringify({name, payload}));
}
```

sendBeacon으로 비동기·비차단 전송. 서버로 모아 분석/대시보드 구성.

#### ④ 알림·롤백 전략 및 헬스체크

모니터링에서 임계치(오류율, 응답시간)를 넘으면 슬랙/메일 알림을 트리거하고, 자동 롤백 정책(이상 징후 시 이전 안정 릴리스로 되돌리기)을 마련하면 가용성을 지킬 수 있으며 사전 헬스체크(Health endpoint)를 통해 배포 전·후 상태를 검증한다.

##### 헬스체크

```
// /health ⇒ 200
app.get('/health', (req, res) ⇒ res.json({status: 'ok', ts: Date.now()}));
```

CI/CD에서 배포 후 이 엔드포인트 검사 후 실제 트래픽 전환(blue/green 또는 canary)을 권장.